Documentação Técnica: Ataque de Profundidade (Nível 70+)

Lógica completa, fórmulas matemáticas e máquina de estados para implementação em jogos 2D

1. Conceito do Ataque (Ilusão de Profundidade 2.5D)

Ao atingir o nível 70, o comportamento da espada familiar abandona o giro lento no próprio eixo e adota uma investida tridimensional simulada. Em um ambiente puramente 2D, essa sensação de que a espada vai para "trás do fundo do cenário" e mergulha em direção à tela é construída pela manipulação sincronizada de quatro propriedades visuais e lógicas:

- **Escala (Tamanho do Sprite):** A redução progressiva da largura e altura simula o afastamento no eixo Z (profundidade).
- **Camada de Desenho (Z-Index / Ordem de Renderização):** Alterar a prioridade de desenho faz a espada passar visualmente por trás de elementos sólidos do cenário (paredes, pilares, árvores ou plataformas).
- **Tonalidade e Brilho (Perspectiva Atmosférica):** Escurecer levemente o sprite quando ele está pequeno reforça a noção de distância e sombra.
- **Controle de Colisão (Hitbox):** Desativar o dano durante o recuo impede que a espada acerte o alvo antes de completar o mergulho.

2. Variáveis de Estado Necessárias

Para programar essa mecânica sem consumir memória extra com novos sprites ou modelos 3D, a entidade da espada precisa gerenciar apenas as seguintes variáveis:

Variável	Tipo	Função na Lógica do Ataque
`estado`	Texto / Enum	Define a fase atual: `SEGUINDO`, `RECUANDO_FUNDO`, `MERGULHANDO` ou `IMPACTO`.
`profundidadeZ`	Decimal (Float)	Varia de `0.0` (plano normal, 100% de tamanho) até `1.0` (fundo máximo, 20% de tamanho).
`alvoX, alvoY`	Decimal (Float)	Armazenam as coordenadas do centro do inimigo travado como alvo.
`recuoX, recuoY`	Decimal (Float)	Ponto deslocado (diagonal acima ou ao lado do alvo) para onde a espada vai enquanto recua.
`hitboxAtiva`	Booleano	Controla se a espada causa dano (`true` apenas no momento exato do impacto).
`timerImpacto`	Decimal (Float)	Tempo curto (em segundos ou frames) em que a espada permanece no plano frontal aplicando dano.

3. Máquina de Estados (Passo a Passo)

Estado 1: Trava de Mira (SEGUINDO → RECUANDO_FUNDO)

- **Detecção:** Quando um inimigo entra no raio de alcance na tela, a espada salva a posição dele em `alvoX` e `alvoY`.
- **Ponto de impulso:** Define um ponto de recuo deslocado (ex: `recuoX = alvoX - 60`, `recuoY = alvoY - 80`) para criar um arco visual bonito.
- **Segurança de colisão:** Define `hitboxAtiva = false` para evitar dano acidental durante a ida para o fundo.

Estado 2: Recuo para Trás do Cenário (RECUANDO_FUNDO)

- **Avanço no eixo Z:** A cada frame, `profundidadeZ` aumenta gradualmente (ex: `+ 2.5 * deltaTime`).
- **Redução de escala:** O tamanho diminui de 100% até 20% conforme `profundidadeZ` se aproxima de `1.0`.
- **Troca de camada (Z-Index):** Quando `profundidadeZ > 0.3`, o `Z-Index` muda para um valor negativo (ex: `-1`), colocando o sprite atrás do cenário principal.
- **Sombra de distância:** A cor do sprite é escurecida proporcionalmente à profundidade (ex: brilho cai de 100% para 45%).
- **Transição:** Ao atingir `profundidadeZ >= 1.0`, trava as coordenadas finais do inimigo e muda para `MERGULHANDO`.

Estado 3: O Mergulho de Alta Velocidade (MERGULHANDO)

- **Retorno acelerado:** `profundidadeZ` cai rapidamente até `0.0` (cerca de 3x mais rápido do que subiu).
- **Crescimento (Zoom In):** A escala volta rapidamente de 20% para 100%, simulando a aproximação brusca na direção da tela.
- **Retorno ao primeiro plano:** Assim que `profundidadeZ < 0.3`, o `Z-Index` volta para a frente de tudo (ex: `+10`) e o brilho da cor retorna a 100%.
- **Rotação direcional:** O ângulo da lâmina aponta diretamente para `(alvoX, alvoY)` enquanto interpola sua posição em linha reta até o alvo.

Estado 4: Impacto e Pós-Imagem (IMPACTO)

- **Ativação do golpe:** Quando `profundidadeZ <= 0.0`, fixa `profundidadeZ = 0.0` e ativa `hitboxAtiva = true`.
- **Efeito de impacto (Pop):** A escala sobe momentaneamente para `1.15` (115%) durante 2 ou 3 frames para dar sensação de peso e impacto.
- **Rastro luminoso (Afterimage):** Desenha 2 ou 3 cópias translúcidas (opacidade 50% e 25%) nas posições anteriores sem criar objetos pesados na memória.
- **Reinício contínuo:** Ao fim do `timerImpacto`, desliga a hitbox. Se o inimigo ainda tiver vida, inicia imediatamente um novo `RECUANDO_FUNDO` sem pausas.

4. Fórmulas Matemáticas Utilizadas

Para garantir que o movimento seja fluido em qualquer taxa de quadros (FPS), utilize interpolação linear baseada em `profundidadeZ` (onde `0.0` é perto e `1.0` é longe):

```
Escala do Sprite (de 1.0 a 0.2):
escala = 1.0 - (profundidadeZ * 0.8)

Brilho da Cor / Sombra (de 1.0 a 0.45):
brilho = 1.0 - (profundidadeZ * 0.55)

Posição Interpolada (Suavização de Trajeto):
posX = lerp(posX, destinoX, velocidade * deltaTime)
posY = lerp(posY, destinoY, velocidade * deltaTime)

Ângulo da Ponta da Espada (em Radianos):
angulo = atan2(alvoY - posY, alvoX - posX)
```

5. Pseudocódigo Completo (Pronto para Adaptar)

Este código funciona na maioria das engines 2D (Godot, Unity, GameMaker, Love2D ou HTML5 Canvas) e mantém o consumo de memória RAM extremamente baixo:

```
// Executado a cada frame do jogo (Update)
funcao atualizarAtaqueEspada(deltaTime):

    se estado == "SEGUINDO":
        hitboxAtiva = falso
        escala = 1.0
        zIndex = 10
        // Lógica normal de flutuar perto do jogador
        alvo = buscarInimigoMaisProximo()
        se alvo != nulo:
            alvoX = alvo.x
            alvoY = alvo.y
            recuoX = alvoX + (escolher(-80, 80))
            recuoY = alvoY - 70
            estado = "RECUANDO_FUNDO"

    senao se estado == "RECUANDO_FUNDO":
        hitboxAtiva = falso
        profundidadeZ = profundidadeZ + (2.5 * deltaTime)

        // Move para a posição de impulso no fundo
        posX = lerp(posX, recuoX, 5.0 * deltaTime)
        posY = lerp(posY, recuoY, 5.0 * deltaTime)

        // Atualiza visual de profundidade
        escala = 1.0 - (profundidadeZ * 0.8)
        brilho = 1.0 - (profundidadeZ * 0.55)

        // Passa para trás do cenário
        se profundidadeZ > 0.3:
            zIndex = -1

        // Aponta para o inimigo enquanto recua
        angulo = atan2(alvoY - posY, alvoX - posX)

        se profundidadeZ >= 1.0:
            profundidadeZ = 1.0
            // Atualiza a posição final do inimigo antes do bote
            se alvo.estaVivo():
                alvoX = alvo.x
                alvoY = alvo.y
            estado = "MERGULHANDO"

    senao se estado == "MERGULHANDO":
        // Volta 3x mais rápido do que recuou
        profundidadeZ = profundidadeZ - (7.5 * deltaTime)

        // Dispara em linha reta até o alvo
        posX = lerp(posX, alvoX, 14.0 * deltaTime)
        posY = lerp(posY, alvoY, 14.0 * deltaTime)

        escala = 1.0 - (profundidadeZ * 0.8)
        brilho = 1.0 - (profundidadeZ * 0.55)
```

```
        // Volta para a frente do cenário
        se profundidadeZ < 0.3:
            zIndex = 10

        se profundidadeZ <= 0.0:
            profundidadeZ = 0.0
            escala = 1.15 // Efeito visual de impacto
            brilho = 1.0
            hitboxAtiva = verdadeiro
            timerImpacto = 0.08 // Duração ativa do golpe (80ms)
            estado = "IMPACTO"

    senao se estado == "IMPACTO":
        timerImpacto = timerImpacto - deltaTime
        escala = lerp(escala, 1.0, 20.0 * deltaTime)

        se timerImpacto <= 0.0:
            hitboxAtiva = falso
            escala = 1.0
            se alvo != nulo e alvo.estaVivo():
                recuoX = alvo.x + (escolher(-80, 80))
                recuoY = alvo.y - 70
                estado = "RECUANDO_FUNDO"
            senao:
                estado = "SEGUINDO"
```

6. Boas Práticas de Otimização para Celular

- **Reaproveitamento de Textura:** Nunca carregue sprites separados para o tamanho pequeno ou para o rastro. Utilize apenas as propriedades de transformação (`scale` e `alpha`) da GPU sobre a mesma imagem carregada na memória.
- **Progressão por Nível:** Você pode usar a mesma lógica para evoluir o ataque conforme o nível da espada sobe no seu jogo, aumentando apenas os multiplicadores de velocidade (2.5 no recuo e 7.5 no mergulho).
- **Sem Alocação Dinâmica de Memória:** Evite instanciar objetos novos para partículas de corte a cada ataque. Mantenha um pequeno array fixo (pool de 3 posições) para desenhar o rastro fantasma da lâmina sem gerar lixo na memória RAM.